Task 1 Byte Pair Encoding algorithm

Initialize corpus

First, define the words and their frequencies. Each word is treated as a separate sequence. Word frequencies = weights:

low: 5

lower: 2

lowest: 3

newer: 3

wider: 2

widest: 3

Turn 0 – Initial tokenization

Split every word into individual characters.

Vocabulary: [d, e, i, l, n, o, r, s, t, w]

Tokenized corpus: [5(l o w), 2(l o w e r), 3(l o w e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Now count all adjacent character pairs. Each pair is weighted by the frequency of ist word.

Turn1 – 1st merge

Pair

low: 5(lo) 5(ow)

lower: 2(lo) 2(ow) 2(we) 2(er)

lowest: 3(lo) 3(ow) 3(we) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

Add the frequencies of identical pairs:

most frequent pairs: 10(lo) 10(ow) 8(we) 7(er) 6(es) 6(st) 5(de) 5(id) 5(wi) 3(ew) 3(ne)

The highest Frequency ist 10. There is a tie between (lo) and (ow).

According to the alphanetical rule (lo) is selected.

Merge 1 : (**l o) -> lo**

Add token to vocabulary and merge the selected pair in every word.

Turn 2 – 2nd merge

Vocabulary: [d, e, i, l, lo , n, o, r, s, t, w] *# vocabulary grows with every new token*

Updated tokenized corpus:

[5(lo w), 2(lo w e r), 3(lo w e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Reinitialize the pair frequency dictionary and count all adjacent pairs again.

Pair frequency calculation:

low: 5(low)

lower: 2(low) 2(we) 2(er)

lowest: 3(low) 3(we) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 10(low) 8(we) 7(er) 6(es) 6(st) 5(de) 5(id) 5(wi) 3(ew) 3(ne)

Highest frequency is 10.

Merge 2: (lo w) -> low

Update the vocabulary and the tokenized corpus (merge).

Turn 3 – 3rd merge

Vocabulary: [d, e, i, l, lo, low, n, o, r, s, t, w]

Updated tokenized corpus:

[5(low), 2(low e r), 3(low e s t), 3(n e w e r), 2(w i d e r), 3(w i d e s t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again using the updated sequences. (# low is finished)

Pair frequency calculation:

lower: 2(lowe) 2(er)

lowest: 3(lowe) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(we) 3(er)

wider: 2(wi) 2(id) 2(de) 2(er)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 7(er) 6(es) 6(st) 5(de) 5(id) 5(low) 5(lowe) 5(wi) 3(ew) 3(ne)

Highest frequency is 7.

Merge 3: (e r) -> er

Update the vocabulary and the tokenized corpus.

Turn 4 - 4th merge

Vocabulary: [d, e, er, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

[5(low), 2(low er), 3(low e s t), 3(n e w er), 2(w i d er), 3(w i d e s t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowe) 3(es) 3(st)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(de) 3(es) 3(st)

most frequent pairs: 6(es) 6(st) 5(id) 5(wi) 3(de) 3(ew) 3(lowe) 3(ne) 3(wer) 2(der) 2(lower)

Highest frequency is 6. Tie betwenn (es) and (st). According to the alphabetical rule (es) is selected.

Merge 4: (e s) -> es

Update the vocabulary and the tokenized corpus.

Turn 5 – 5th Merge

Vocabulary: [d, e, er, es, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

[5(low), 2(low er), 3(low es t), 3(n e w er), 2(w i d er), 3(w i d es t)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowes) 3(est)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(des) 3(est)

most frequent pairs: 6(est) 5(id) 5(wi) 3(des) 3(ew) 3(lowes) 3(ne) 3(wer) 2(der) 2(lower)

highest frequency is 6.

Merge 5: (es t) -> est

Update the vocabulary and the tokenized corpus.

Turn 6 – 6th Merge

Vocabulary: [d, e, er, es, est, i, l, lo, low, n, o, r, s, t, w]

updated tokenized corpus:

tokenized [5(low), 2(low er), 3(low est), 3(n e w er), 2(w i d er), 3(w i d est)]

Reinitialize the pair frequency dictionary. Count all adjacent pairs again

Pair frequency calculation:

lower: 2(lower)

lowest: 3(lowest)

newer: 3(ne) 3(ew) 3(wer)

wider: 2(wi) 2(id) 2(der)

widest: 3(wi) 3(id) 3(dest)

most frequent pairs: 5(id) 5(wi) 3(dest) 3(ew) 3(lowest) 3(ne) 3(wer) 2(der) 2(lower)

Higehest frequency is 5. Tie betwenn (id) and (wi). According to the alphabetical rule (id) is selected.

Merge 6: (i d) -> id

Update the vocabulary and the tokenized corpus.

Final Result

Final Vocabulary: [d, e, er, es, est, i, id, l, lo, low, n, o, r, s, t, w]

finalized tokenized corpus:

[5(low), 2(low er), 3(low est), 3(n e w er), 2(w id er), 3(w id est)]

Six merge operations:

1. l o -> lo
2. lo w -> low
3. e r -> er
4. e s -> es
5. es t -> est
6. i d -> id

Tokenize unseen word *lowers*

Initialize on character level

[l o w e r s]

Apply the learned BPE Merges in order

1. l o -> lo
2. lo w -> low
3. e r -> er
4. ! e s -> es not possible because *e* is no longer a seperate token. It already belongs to *er*.
5. ! e s t -> est not possible
6. ! i d -> id not possible

Final tokenization: [low er s]

<u>Reflection:</u>

(explanation: Downstream NLP means here applying learned tokenizer of an unseen word (lowers) after the upstream training phase with the corpus)

- The learned tokenization partly reflects the morphologically structure of the words.
- It works for [low], [low er], [low est] and [low er s] where *low* represents the word stem (lexical/semantic meaning) and *er, est, s* are meaningfull suffixes (grammatical meaning) – they are all "real morphemes" althoug they where only merged by counting frequencies.
- For [n e w er], [w id er] and [w id est] the tokenization only partly refelcts meaningful morpheme structures. Here, only the tokens *er* and *est* are meaningful suffixes. *new* is not merged into one token and *id* in *wider* and *widest* has no linguisitic meaning.
- Meaningful morphemes can be useful for downstream NLP tasks because they can help understand word meanings and grammatical forms.
- However BEP does not always recognize true morphemes since it merges tokens based on frequency rather than meaning. This can result in negativ/false performance in downstream NPL tasks.
- For example with the word [w id er] the model may need more training examples to learn the meaningful connection between the tokens or it might also have difficulty recognizing the word's meaning. In a text classification task this could lead to lower accuracy.